

Agent 入口高并发技术方案

针对正常 Agent 入口 `POST /api/playground/run` (含 `resume`)：对话仍在请求路径上返回；用限流、舱壁、熔断挡住下游；Studio / Langfuse / 重 Trace 进异步队列；Agent、Skill、MCP、模型配置走多级缓存。

文档类型：评审用设计稿（尚未改代码） 范围：调试台 Agent 入口，不含 RAG ingest 日期：2026-09-09

原则：用户等的是这一轮回复

把整次 `generate_chat_reply` 丢进 Celery 只会让调试台变慢、会话状态更难续跑。队列只承接「用户已经拿到回复之后」的工作。

1. 现状瓶颈

当前一轮调试是同步串起来的：鉴权 → 查 Agent / Skill / MCP → `complete_chat` (httpx 最长 45s, 最多 4-8 轮工具循环) → 写会话 → 等 Studio 导出完才返回。评测是单 daemon 线程每 1.2s 扫 MySQL。高并发时，慢的是模型，卡死的是 uvicorn 线程和观测导出。

同步占满

LLM 45s 阻塞 uvicorn 线程

1 线程

评测工人轮询 MySQL

无共享配额

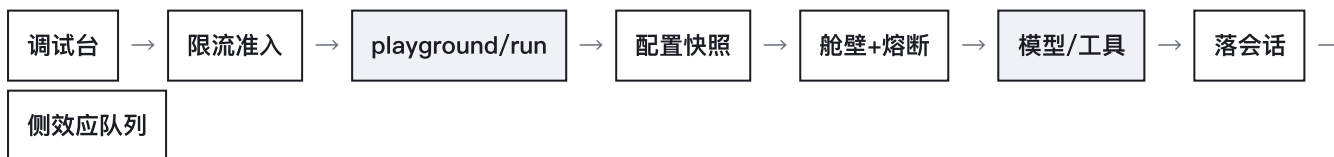
租户 / 模型没有并发上限

侧效应同步

Studio 导出卡在返回前

2. 目标请求路径

热路径：准入 → 读缓存快照 → 占一个 Agent / 模型槽位 → 调模型与工具 → 写会话后立刻返回。冷路径：观测导出与评测。



2.1 一次 playground/run 时间花在哪

典型一轮（有密钥、1-2 次模型调用）· 估算占比，非压测数据

72% 模型调用 · 12% 工具 · 8% Studio 导出 · 5% 会话落库 · 3% 读 Agent / Skill / MCP

2.2 三类流量分开，不要抢同一条管道

入口	用户是否在等	策略
POST /api/playground/run resume	是，要这一轮回复	同步或 SSE；限流 + 舱壁 + 熔断；配置走缓存
离线评测 EvaluationRun	否	独立 eval 队列与 worker，禁止占用 playground 槽位
Studio / Langfuse / 仪表盘汇总	否	observe 队列，失败可丢可重试，绝不能拖住 JSON 返回

2.3 建议组件（生产最小集）

Redis 共享状态 令牌桶计数、熔断器状态、L2 缓存、队列 broker。多 uvicorn worker 必须共享，进程内字典不够。	FastAPI 热路径 鉴权、准入、读快照、调模型、写 Conversation / ChatMessage。 complete_chat 改异步 httpx，避免一个 45s 调用占死线程。	Worker 冷路径 ARQ 或 Celery： studio.export、 langfuse.export、 eval.run、mcp.refresh。业务状态仍以 MySQL 为准。
---	---	---

3. 熔断与限流

3.1 限流：四层漏斗，专打 Agent 入口

只做全局限流会被一个租户或一个慢模型拖死。playground 按租户、用户、Agent 并发、模型供应商四层扣配额，先满先拒，返回 429 并带 Retry-After。

层	键	默认建议	挡住什么
租户 QPS	tenant_id	30 req/s，突发 60	某个租户刷调试台
用户 QPS	user_id	5 req/s，突发 10	单人连点发送
Agent 在途	agent_id 信号量	每 Agent 8 路	同一 Agent 把模型配额打满
模型供应商	provider + model_id	按密钥档位，如 20 在途	触发 DashScope / OpenAI 429

3.2 舱壁（bulkhead）

playground、eval、sandbox、browser 四个池子，互不借用。浏览器工具现在是进程级一把锁，高并发下应改为「每租户 1 个浏览器会话」，不要让评测和打开网页堵住对话。

3.3 熔断：按下游，不要按整站

下游	打开条件	打开时入口行为	半开
LLM complete_chat	连续 5 次超时 / 5xx / 429, 或 30s 错误率 > 50%	立即失败, 返回已有检查点 + 可 resume; 不打预览假成功	放 1 路试探, 成功则闭合
单个 MCP HTTP	连续 3 次失败或超时	该工具从本轮 tools 列表摘掉, 其它工具继续	后台 mcp.refresh 探测成功后恢复
沙箱执行	超时 / 后端不可用	本轮拒绝 sandbox_*, 对话仍返回	probe_sandbox 成功后恢复
Studio / Langfuse	连续失败	请求路径直接跳过, 任务进死信或丢弃	观察队列自动重试

失败语义要和现有检查点对齐

generate_chat_reply 已会把 running / failed 写入 workspace checkpoint。熔断触发时应走同一套：前端仍可点续跑，不要另造一套「降级预览」冒充模型已调用。

4. 异步任务队列

现状：评测是单 daemon 线程每 1.2s SELECT queued；Studio 导出在 _finalize_playground 里同步 httpx。高并发下这两处会直接拉长 Agent 入口 P99。

队列	任务	何时入队	并发
observe	studio.export / langfuse.export	会话 commit 成功之后	高，可丢，重试 3 次
eval	evaluation.execute	创建离线回归时	低，按模型舱壁限流，与 playground 隔离
refresh	mcp.probe / sandbox.probe / dashboard.rollups	定时或配置变更	中，写回 L2 缓存

4.1 入口返回前必须做完的

写 ChatMessage、Conversation、本地 checkpoint、给前端的 reply / session_id / spans。这些失败应让请求失败，不能「先返回再补写会话」，否则 resume 会对不上。

4.2 可以延后的

persist_run 里非检查点的大文件、Studio OTLP、Langfuse、仪表盘计数、MCP 工具列表刷新。入口只投递 job_id，UI 轮询或忽略均可。

评测不要再 poll MySQL

`_claim_run` 无 `SELECT FOR UPDATE`，多进程会抢同一条。改为 Redis 队列，`EvaluationRun` 只作状态机；worker 用和 playground 同一套 `generate_chat_reply`，但走 eval 舱壁。

5. 多级缓存

Agent 回复几乎每次都不同，默认不要缓存 `chat/completions`。要缓存的是入口每次都打库的那几样：Agent 绑定、Skill 正文、MCP 工具 schema、模型端点、RBAC。

级	介质	存什么	TTL / 失效
L1	进程内	当前用户、角色权限、Agent 快照、Skill 正文、MCP tools JSON	30—60s；写路径 pub/sub 立刻删
L2	Redis	上述快照跨进程、MCP probe 结果、仪表盘汇总、熔断器 counters	2—5 min；配置保存时主动删
L3	MySQL	事实来源：Agent / Skill / Model / Conversation	不设 TTL；会话与检查点仍以库和 workspace 为准

5.1 入口读路径（命中时）

playground/run 不再 `selected_skills` + `skill_instruction` + `list_mcp_tools` 连环查库，而是 `get_agent_runtime_snapshot(agent_id)`：系统提示已拼好、tools schema 已备好、模型 `base_url` 已解析。密钥不进 Redis 明文，L1 只留「是否就绪」和从环境变量现取。

要缓存

`dump(agent)` 绑定列表、SKILL.md 正文、MCP 工具名与 `parameters`、沙箱策略、实验分流结果（同一 `session_id`）、dashboard 聚合。

不要缓存

模型生成结果、工具执行输出、API Key、checkpoint 工作集（正在跑的 `pending_tools`）。会话消息可短 TTL 热读，写入仍以 MySQL 为准。

6. 落地节奏

期	改动	入口立刻获得什么	完成标准
P0 本进程	<code>asyncio httpx</code> ；租户/用户令牌桶；LLM / MCP / Studio 熔断；Studio 改 <code>fire-and-forget</code> 线程	慢导出不再拖 P99；模型挂了快速失败	单机压测 playground 在途数可控，Studio 挂掉不影响 200

期	改动	入口立刻获得什么	完成标准
P1 Redis	共享限流与熔断；L2 配置快照；observe + eval 真队列	多 worker 配额一致；评测不再抢对话框	两进程 uvicorn 限流不翻倍；eval 与 chat 隔离
P2 流式与配额面板	playground SSE / stream；管理平台看到熔断状态与队列积压；按租户配额配置	首 token 更快；运营能看到谁被限	前端能续跑；429 / 熔断有明确文案

6.1 明确不做

- 不把整次对话变成「提交任务再轮询结果」——现有 checkpoint / resume 已经覆盖失败续跑。
- 不上服务网格也能先做应用内熔断。
- 不把 FAISS / RAG 塞进这条入口，那是另一条 ingest 链路。

若方案可接受，第一刀建议

拆 `_finalize_playground`：会话与 Trace 仍同步 commit；`export_playground_to_studio` 与 Langfuse 改为入队。同时给 `complete_chat` 加按 provider 的熔断和在途信号量。这两处改动面最小、对入口收益最大。

源：AgentForge Control Plane · playground/run → generate_chat_reply → complete_chat (httpx 45s) → _finalize_playground。
本文档由评审设计稿导出，供内部传阅；实施前以评审结论为准。